

Got a problem statement for which we need to build a website for real-life usecase that can be easily used by non technical users, handles realistic values and scenarios & could be easily deployed and without major changes (it's actually for a promptathon).

For now, do not generate the code, just think and evaluate about all the needs, requirements, architecture, design, ui, ux, frontend, backend, data management, security, monitoring etc.

Show more ▾



This is exactly the kind of problem where many teams will build a working calculator, but very few will build a **real product**. For a Promptathon, judges will usually reward the team that thinks beyond the formula and demonstrates product thinking, architecture quality, edge-case handling, UX polish, and deployability.

1. Understanding What The Judges Actually Want

The obvious solution is:

- Enter bill
- Enter people
- Select tip
- Split amount
- Show result

Everyone will do that.

The winning solution is:

"A production-ready expense sharing platform that real people can use immediately without confusion."

The difference is huge.

2. Real Life User Personas

Think about who will use it.

Persona 1: Friends at a Restaurant

Bill = ₹2,430

People:

- Ashutosh
- Rahul
- Amit

Tip = Good (15%)

Need quick split.

Persona 2: Flatmates

Monthly expenses:

- Rent
- Electricity
- WiFi
- Groceries

Need history and tracking.

Persona 3: Unequal Contributions

Example:

Bill = ₹1000

Person A = 40%

Person B = 35%

Person C = 25%

Need exact calculation.

Persona 4: One Person Already Paid

Rahul paid full bill.

Need to know:

- Who owes Rahul?
- How much?

This is a very realistic enhancement.

3. Product Vision

Instead of:

Bill Splitter

Think:

Smart Bill Splitter & Shared Expense Manager

Tagline:

Split expenses fairly, accurately and without arguments.

Judges love product positioning.

4. Core Modules

Module A — Expense Entry

Input:

- Expense Name
- Amount
- Date
- Category

Examples:

- Rent
- Food
- Electricity
- Internet

Validation:

- Amount > 0
- No negative values

Module B — People Management

Add participants.

Example:

Name

Ashutosh

Rahul

Amit

Allow:

- Add
- Remove
- Edit

Module C — Split Strategy Engine

This is the heart.

Support:

Equal Split

$1000 \div 4$

Percentage Split

Ashutosh = 40%

Rahul = 35%

Amit = 25%

Must validate:

Total = 100%

Custom Amount Split

Ashutosh = ₹400

Rahul = ₹350

Amit = ₹250

Must validate:

Sum = Bill Amount

5. Tip Engine

Required by problem statement.

Options:

Quality	Tip
Poor	5%
Good	15%
Excellent	20%

Example:

Bill = ₹1000

Good = ₹150

Total = ₹1150

Then split.

Important:

Tip must be applied BEFORE splitting.

6. Critical Edge Case (Most Important)

The judges specifically highlighted this.

Example:

₹100

3 people

Raw:

$100/3$

=

33.33333333...

Wrong.

Correct:

Person A = 33.33

Person B = 33.33

Person C = 33.34

Total:

33.33

+

33.33

+

33.34

=

100.00

Perfect.

Production Approach

Never calculate using floating-point money.

Store everything as:

Paise/Cents

₹100

=

10000 paise

Split:

$10000 / 3$

=

3333 remainder 1

Give:

3333

3333

3334

Convert back:

33.33

33.33

33.34

This is how financial systems work.

Judges will LOVE this.

7. UX Design

Most teams ignore UX.

Big mistake.

Step-Based Flow

Step 1

Add Expense

Step 2

Add People

Step 3

Choose Split Method

- Equal
- Percentage
- Custom

Step 4

Choose Tip

- Poor
- Good
- Excellent

Step 5

Review

Step 6

Results

Very intuitive.

8. Results Screen

Instead of a boring table:

Show cards.

Example:

 Ashutosh

₹460.00

 Rahul

₹345.00

 Amit

₹345.00

Also show:

Bill = ₹1000

Tip = ₹150

Final = ₹1150

This improves judge perception instantly.

9. Advanced Real-Life Enhancements

These are not required but score points.

Expense History

Store previous expenses.

Example:

May 2026

- Rent
- Electricity
- Food

Monthly Analytics

Show:

Total spent

Category breakdown

Top expense category

Export

Download:

- PDF
- CSV

Judges love exports.

Share Result

Generate shareable summary.

Example:

Expense Summary

Bill: ₹1000

Tip: ₹150

Ashutosh: ₹460

Rahul: ₹345

Amit: ₹345

10. Frontend Architecture

Keep modular.

pages/

└─ Dashboard

└─ ExpenseForm

└─ Results

└─ History

```
components/  
├─ PersonCard  
├─ ExpenseCard  
├─ TipSelector  
├─ SplitMethodSelector  
└─ SummaryCard
```

```
services/  
├─ splitEngine  
├─ tipCalculator  
└─ validationService
```

Very scalable.

11. Backend Architecture

Even if simple.

```
API Layer  
  ↓  
Business Logic Layer  
  ↓  
Data Layer
```

Business Logic:

- Split calculations
- Tip calculations
- Validation
- Rounding engine

Keep calculation logic separate from UI.

Judges often check this.

12. Data Model

Expense

```
Expense  
- id  
- title  
- amount  
- tipPercentage
```

- totalAmount
- createdAt

Person

- Person
- id
 - name

Share

- Share
- personId
 - amount
 - percentage

13. Validation Rules

Important.

Bill:

✗ Empty

✗ Negative

✗ Zero

Percentage Split

Must equal:

100%

Custom Split

Must equal:

Bill Total

Names

No duplicates.

14. Security Considerations

Even for a small project.

Input Sanitization

Prevent invalid entries.

Rate Limiting

If backend exists.

Secure Validation

Validate on both:

- Frontend
- Backend

No Client Trust

Never trust frontend calculations.

Backend rechecks.

15. Monitoring

Most teams won't think about this.

You should.

Track:

- Failed calculations
- Validation errors
- API failures

Display:

Friendly messages.

Not:

```
Undefined is not a function
```

16. Deployment Strategy

Judges like deployable products.

Simple stack:

Frontend:

- React

Backend:

- Node.js

Database:

- SQLite (simple)
or
- PostgreSQL

Deployment:

- Frontend → Vercel
- Backend → Render

Easy demo.

17. Judge Scoring Perspective

If I were judging:

Area	Weight
Correctness	25%
Edge Case Handling	20%
UX/UI	15%
Real World Practicality	15%
Architecture	10%
Deployability	5%
Security & Validation	5%
Innovation	5%

The **decimal handling engine** is the most important technical part because the problem statement explicitly emphasizes it.

What Would Make This Submission Stand Out?

Add these 5 things:

- ✅ Financial-grade rounding using paise/cents
- ✅ Multiple split methods (equal, percentage, custom)
- ✅ Expense history
- ✅ Responsive mobile-first UI
- ✅ "Who owes whom" settlement summary

At that point you're no longer submitting a calculator—you are submitting a realistic, deployable expense-sharing product that directly addresses the stated requirements while demonstrating strong software engineering and product design thinking. 🚀

Pasted text.txt
Document

index.html

File

review it and give the prompts for remaining for the
script.js

index(1).html
File

script.js
JavaScript

style.css
File

see the website as a judge , review everything take a note
of all the bugs non-responsive actions and all the logic

working or not , the look is fine the problem here is that its
not usable for now, sort everything suggest the solutions

index(2).html

File

script(1).js

JavaScript

ok now take your time analyze both and we'll discuss further only after getting to the updates file which would be working well, also keep in mind to make the changes in such a way that css don't get altered

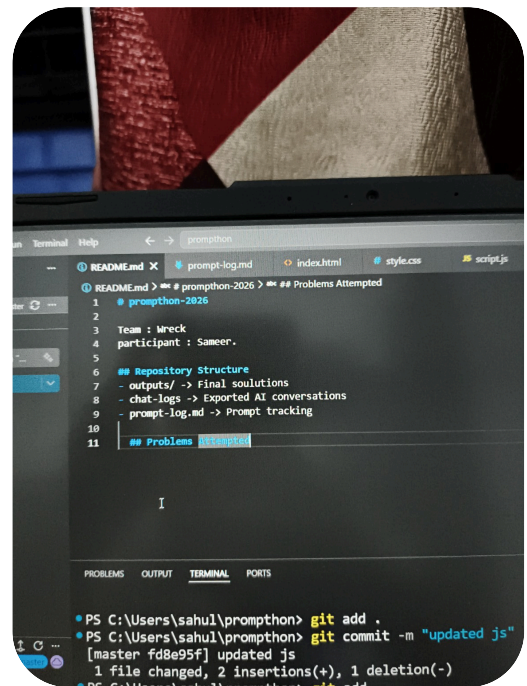
script(3).js
JavaScript

script(4).js
JavaScript

script(5).js
JavaScript

index(3).html
File

Make it in this format



I mean I need to paste it here so give the content accordingly

If you're planning to submit **many projects**, then I would slightly improve the structure now before it grows.

Your current README structure is fine for 1–2 projects, but after 10+ projects it becomes hard to scan.

I'd recommend:

Markdown

prompthon-2026

Team: Wreck

Participant: Sameer

Repository Structure

- outputs/ → Final solutions
- chat-logs/ → Exported AI conversations
- prompt-log.md → Prompt tracking

Problems Attempted

Problem	Title	Status	Solution
A1	Smart Bill Splitter & Expense Tracker	✅ Completed	outputs/A1
A2	Coming Soon	🚧 In Progress	outputs/A2
A3	Coming Soon	🚧 In Progress	outputs/A3

A1 - Smart Bill Splitter & Expense Tracker

Overview

Frontend-only expense splitting and tracking application built using:

- HTML
- CSS
- Vanilla JavaScript

Features

- Equal Split
- Percentage Split
- Custom Amount Split
- Tip Calculation
- Expense History
- Dashboard Analytics
- Responsive UI

Files

```
```text
outputs/
├─ A1/
│ ├── index.html
│ ├── style.css
│ └─ script.js
```

## Documentation

- [prompt-log.md](#)
- [chat-logs/](#)

---

### ### Why this is better

When you eventually have:

```
```text
A1
A2
A3
A4
A5
A6
...
```

a judge can immediately see:

Problem	Status
A1	✓
A2	✓
A3	✓

without scrolling through long descriptions.

Then each project gets its own section below.

I would also strongly recommend this folder structure:

```

outputs/
├─ A1/
│   ├─ index.html
│   ├─ style.css
│   └─ script.js
│
├─ A2/
│   └─ ...
│
└─ A3/
    └─ ...

```

This scales much better and makes judging easier because every problem is isolated and easy to locate. 🚀

📄 ⬆️ ↺️ ⋮ 📖 Sources

Ok everything is fine now just add something in problems attempted section, make the column set for all problems (A1 to A7, B1 to B4) I'll update later accordingly.

You can paste this directly under **## Problems Attempted** and update the status/details later:

Status Legend

Problem	Title	Status	Solution Folder
A1	Smart Bill Splitter & Expense Tracker	✅ Completed	outputs/A1
A2	TBD	⌚ Pending	outputs/A2
A3	TBD	⌚ Pending	outputs/A3
A4	TBD	⌚ Pending	outputs/A4
A5	TBD	⌚ Pending	outputs/A5
A6	TBD	⌚ Pending	outputs/A6
A7	TBD	⌚ Pending	outputs/A7
B1	TBD	⌚ Pending	outputs/B1
B2	TBD	⌚ Pending	outputs/B2
B3	TBD	⌚ Pending	outputs/B3
B4	TBD	⌚ Pending	outputs/B4

Status Legend

- ✅ Completed
- 🚧 In Progress
- ⌚ Pending
- ❌ Not Attempted

This scales nicely because when you're done you can simply change rows like:

Markdown

```
| A2 | Project Name | ✅ Completed | outputs/A2 |
| B1 | Project Name | 🚧 In Progress | outputs/B1 |
```

without restructuring the README. It also gives judges a quick overview of every attempted problem at a glance.

Provide in markdown style

Markdown

Problems Attempted

Problem	Title	Status	Solution Folder
A1	Smart Bill Splitter & Expense Tracker	✅ Completed	outputs/A1
A2	TBD	⌚ Pending	outputs/A2
A3	TBD	⌚ Pending	outputs/A3
A4	TBD	⌚ Pending	outputs/A4
A5	TBD	⌚ Pending	outputs/A5
A6	TBD	⌚ Pending	outputs/A6
A7	TBD	⌚ Pending	outputs/A7
B1	TBD	⌚ Pending	outputs/B1
B2	TBD	⌚ Pending	outputs/B2
B3	TBD	⌚ Pending	outputs/B3
B4	TBD	⌚ Pending	outputs/B4

Status Legend

- ✅ Completed
- 🚧 In Progress
- ⌚ Pending
- ❌ Not Attempted

If you want it to look even cleaner for judges, you can later replace TBD with the actual problem titles while keeping the same table structure.

